

A Project Report
on

A Visual Simulator for Turing Machine Computations: Unary Addition and Multiplication

Submitted by

AKASH KHEDAR [Reg No: RA2311026030125]
DHURVA SINGH [Reg No: RA2311026030126]
SATYA PRAKASH PATNAIK [Reg No: RA2311026030127]

Under the Supervision of
Ms. Akansha Bisht
(Assistant Professor, Department of Computer Science Engineering)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING & TECHNOLOGY
SRM INSTITUTE OF SCIENCE & TECHNOLOGY
DELHI-NCR CAMPUS, MODINAGAR
SIKRI KALAN, DELHI-MEERUT ROAD,
DIST. GHAZIABAD – 201204

Odd Semester – 2025

SRM INSTITUTE OF SCIENCE TECHNOLOGY

(Under Section 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified that this case study on “**A Visual Simulator for Turing Machine Computations: Unary Addition and Multiplication**”, is the bonafide work of Akash Khedar [RA2311026030125], Dhruva Singh [RA2311026030126] and Satya Prakash Patnaik [RA2311026030127] who carried out the case study under my supervision. Certified further, that to the best of my knowledge, the work reported here does not form any other project report or dissertation.

SIGNATURE

Ms. Akansha Bisht
Assistant Professor
Computer Science and Engineering

SIGNATURE

Dr. Harish Kumar
Head of Department
CSE Department

Acknowledgment

We would like to express our special thanks and deepest gratitude to our teacher, **Ms. Akansha Bisht** as well as our HOD CSE Department, **Dr. Harish Kumar** for granting us the golden opportunity to undertake this case study on the topic “**A Visual Simulator for Turing Machine Computations: Unary Addition and Multiplication**”, which helped us in doing some research work and hence we came to know about many new concepts. We are thankful to them.

Secondly, we would also like to thank my parents and friends who helped us in the completion of this project within the stipulated time limit.

Table of Contents

S. No.	Content	Page No.
1.	Section 1: Introduction to the Turing Machine as a Model of Computation	1-2
2.	Section 2: Theoretical Foundations and Formalism	3-4
3.	Section 3: Algorithmic Design for Unary Arithmetic	5-8
4.	Section 4: System Architecture and Implementation	9-11
5.	Section 5: Verification and Testing	12-13
6.	Section 6: Evaluation and Future Work	14-15
7.	Section 7: Conclusion	16
8.	References	17

Section 1: Introduction to the Turing Machine as a Model of Computation

1.1 Historical Context and Alan Turing's Contribution

The Turing machine, a cornerstone of modern computer science, was not conceived as a blueprint for a physical computer but as a profound intellectual tool to address a foundational crisis in mathematics. In the early 20th century, mathematician David Hilbert posed the *Entscheidungsproblem*, or "decision problem," which asked if an algorithm exists that can determine whether any given statement in first-order logic is provable from a set of axioms.¹ To answer this, one first needed a rigorous, formal definition of what constitutes an "algorithm" or a "mechanical procedure".²

In his seminal 1936 paper, "On Computable Numbers, with an Application to the Entscheidungsproblem," British mathematician Alan Turing introduced a theoretical device he called an "a-machine" (automatic machine).² This abstract machine, consisting of a simple read/write head operating on an infinite tape, was designed to capture the essence of human computation—the process of a person following a fixed set of rules to manipulate symbols.² It was Turing's doctoral advisor, Alonzo Church, who later reviewed the paper and coined the term "Turing machine".² Turing used this model to prove that no such general algorithm for the *Entscheidungsproblem* could exist, thereby demonstrating that certain mathematical problems are undecidable. This act not only answered Hilbert's question but also laid the groundwork for the entire field of computability theory.

1.2 The Church-Turing Thesis and the Concept of Computability

Concurrent with Turing's work, Alonzo Church developed a different formal system called lambda calculus. Both Turing and Church independently arrived at the same negative answer to the *Entscheidungsproblem*. The remarkable fact that their vastly different formalisms—one a mechanical model, the other a system of functions—were computationally equivalent led to the formulation of the Church-Turing thesis.²

The thesis is not a mathematical theorem that can be proven, but rather a powerful hypothesis about the nature of computation. It posits that any function that can be considered "effectively calculable" by an intuitive, algorithmic process can be computed by a Turing machine.² The continued equivalence of every new, reasonable model of computation proposed since (such as register machines and recursive functions) has provided overwhelming evidence for the thesis. This establishes the Turing machine as a universal model of computation, providing a stable, formal framework to reason about the capabilities and, more importantly, the inherent limitations of any digital computer.⁴

1.3 Project Scope and Objectives

This project aims to bridge the gap between the abstract theory of the Turing machine and its practical, observable execution. While the Turing machine was instrumental in proving the existence of uncomputable functions like the Halting Problem, this report focuses on problems that are demonstrably computable: basic arithmetic. The project's objectives are twofold:

1. To design and formally document the algorithms for two fundamental arithmetic operations—unary addition and unary multiplication. This process will demonstrate how complex tasks can be systematically decomposed into the primitive operations of symbol manipulation that a Turing machine can perform.
2. To implement these algorithms within a C++ application featuring a graphical user interface (GUI). This application will function as a pedagogical tool, a "visualizer," that allows a user to observe the step-by-step execution of the Turing machine. By rendering the machine's tape, head position, and current state in real-time, the visualizer makes the abstract computational process tangible and intuitive.

Ultimately, this project explores the boundary first drawn by Turing. By simulating computable functions, we gain a deeper appreciation for the mechanics of computation itself. This understanding, in turn, provides the necessary context to comprehend the profound implications of those problems that lie beyond the machine's—and thus any computer's—reach.

Section 2: Theoretical Foundations and Formalism

2.1 The 7-Tuple Formal Definition

A deterministic Turing machine (TM) is defined as a 7-tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$, where each component has a specific role.⁶

- Q is a finite, non-empty set of **states**. These represent the machine's internal memory or "state of mind" at any given moment.² For the arithmetic machines in this project, states will correspond to distinct phases of the algorithm, such as "scanning for the separator" or "copying a digit."
- Σ is the **input alphabet**, a finite set of symbols that are allowed to appear on the tape initially. For this project's unary addition, $\Sigma = \{1, +\}$; for multiplication, $\Sigma = \{1, *\}$. The blank symbol is explicitly not part of the input alphabet.⁷
- Γ is the **tape alphabet**, a finite set of symbols that the machine's head can read and write. It is a superset of the input alphabet, $\Sigma \subseteq \Gamma$. It must contain the blank symbol B and may include auxiliary symbols used during computation. For multiplication, for instance, we will use markers like 'X' and 'Y' to keep track of processed digits, so $\Gamma = \{1, *, B, X, Y\}$.⁸
- δ is the **transition function**, which effectively serves as the machine's program.⁴ It is a partial function that takes the current state and the symbol being scanned by the head as input and returns a new state, a new symbol to write on the tape, and a direction for the head to move. Its signature is $\delta: Q * \Gamma \rightarrow Q * \Gamma * \{L, R\}$, where L denotes a move to the left and R a move to the right.³
- q_0 in Q is the designated **start state**, the state in which the machine begins its computation.
- B in Γ is the **blank symbol**. It is the only symbol that can appear infinitely often on the tape at any step of the computation.²
- $F \subseteq Q$ is the set of **final or accepting states**. If the machine enters any state in F , it immediately halts and is said to have accepted its input.⁶ An alternative halting condition occurs if the machine is in a configuration for which is not defined.³

2.2 Anatomy of a Turing Machine

The formal definition is realized through three conceptual components:

- **The Infinite Tape:** This is the machine's memory, conceptualized as a one-dimensional strip of discrete cells, unbounded in both directions.² Each cell holds exactly one symbol from the tape alphabet. The assumption of an infinite tape ensures that the machine's computational power is not limited by memory constraints, only by the algorithm itself.² In any practical simulation, this "infinity" is handled by making the tape arbitrarily extendable as needed.²
- **The Read/Write Head:** This is the active component that interfaces with the tape. At any given time, the head is positioned over a single cell. In one discrete step, it can perform three actions: read the symbol in the current cell, write a new symbol (potentially the same

one) into that cell, and move one cell to the left or right.² This simple, localized mechanism is the machine's only means of accessing and manipulating its memory.

- **The State Control:** This is a finite automaton that embodies the machine's logic. It stores the current state q in Q and contains the definition of the transition function.² Based on the current state and the symbol read by the head, the state control dictates the machine's next action.

2.3 Machine Configuration and Computation

The complete status of a Turing machine at any point in time is its **configuration**, also known as an Instantaneous Description (ID). A configuration can be formally represented as a triple (x, q, k) , where x is the current string on the tape, q is the machine's current state, and k is the integer position of the head.⁴

A single computational **step** (or move) is the process of transitioning from one configuration to another, as dictated by the transition function. A **computation** is a sequence of configurations, starting from an initial configuration and proceeding step-by-step. The computation halts if the machine enters a final state or a configuration for which no transition is defined.¹⁰ The final content of the tape upon halting is considered the output of the computation.

2.4 Unary Number Representation

Modern computers perform arithmetic using binary (base-2) representation. However, for a Turing machine, unary (base-1) representation is often preferred for pedagogical clarity and algorithmic simplicity.¹² In unary, a non-negative integer n is represented by a sequence of n instances of a single symbol, typically '1'.¹⁴ For example, the number 3 is represented as 111, and 5 is 11111.

This choice is not arbitrary; it is a direct consequence of the Turing machine's minimalist design. The machine lacks a built-in Arithmetic Logic Unit (ALU) and can only perform the most primitive symbol manipulations: writing, moving, and changing state.³ While binary addition is possible, it requires a more complex algorithm to handle carries and positional values.¹⁵ Unary addition, in contrast, can be reduced to a simple problem of string concatenation. The operation $3+2$, represented as 111+11, can be solved by manipulating these symbols to produce 11111.¹⁴ This simplification elegantly exposes the fundamental nature of TM computation: the reduction of abstract problems to a sequence of basic, mechanical steps.

Section 3: Algorithmic Design for Unary Arithmetic

3.1 Unary Addition (m+n)

The goal is to compute the sum of two unary numbers, m and n , given an initial tape configuration of $1^m + 1^n$. The final tape should contain $1^{(m+n)}$.

3.1.1 Algorithm Description

A simple and intuitive algorithm achieves this by effectively concatenating the two strings of '1's.

1. **Find Separator:** Starting from the leftmost symbol, the head moves right until it finds the $+$ separator.
2. **Overwrite Separator:** The head replaces the $+$ symbol with a 1. This action conceptually adds the first '1' of the second number to the first number's block.
3. **Find End:** The head continues moving right, past all the '1's of the second number, until it encounters the first blank symbol (B), signifying the end of the input.
4. **Cleanup:** The head moves one cell to the left and erases the final 1 by replacing it with a blank symbol. This step is necessary because overwriting the $+$ in step 2 resulted in a total of $m+n+1$ ones.
5. **Halt:** The machine enters a halting state. The tape now correctly contains a single block of $m+n$ ones.

3.1.2 Formal Transition Table

The algorithm described above is formally captured by the transition function detailed in Table 3.1. The machine starts in state q_0 .

Table 3.1: State Transition Table for Unary Addition

Current State (Q)	Read Symbol (Γ)	Write Symbol (Γ)	Move ({L,R})	Next State (Q)	Description
q_0	1	1	R	q_0	Scan right, looking for the separator.
q_0	$+$	1	R	q_1	Found separator, replace with '1', move to next phase.
q_1	1	1	R	q_1	Scan right, looking for the end of the second number.

q_1	B	B	L	q_2	Found end of input, move left to erase the extra '1'.
q_2	1	B	L	q_f	Erase the last '1' and halt.

3.2 Unary Multiplication ($m * n$)

Multiplication is fundamentally a process of repeated addition. To compute $m * n$, we add the number n to itself m times. The Turing machine algorithm for this is significantly more complex, requiring a nested loop structure and auxiliary symbols to mark progress.⁹ The input is assumed to be in the format $1^m * 1^n$.

3.2.1 Algorithm Description

1. **Outer Loop Initialization:** The machine scans to the first '1' of the multiplier (m) and replaces it with a marker symbol, X , to indicate that one iteration of the outer loop has begun.
2. **Find Multiplicand:** The head moves right, past any remaining '1's of the multiplier and the $*$ separator, to find the beginning of the multiplicand (n).
3. **Inner Loop (Copying):**
 - o The machine iterates through the multiplicand. For each '1' it finds, it replaces it with a temporary marker, Y .
 - o It then scans to the far right of the tape, past all existing symbols, and writes a new 1 in the first blank cell. This effectively copies one digit of the multiplicand to the result area.
 - o The head then returns left to the Y it just wrote and continues scanning right to find the next '1' in the multiplicand.
4. **Reset Multiplicand:** Once all '1's in the multiplicand have been changed to Y s (and thus copied), the machine scans back left through the Y s, changing them all back to 1s. This resets the multiplicand for the next iteration of the outer loop.
5. **Repeat Outer Loop:** The head moves back to the left of the $*$ separator to find the next X and then the next '1' of the multiplier. It replaces this '1' with an X and repeats the entire inner loop process.
6. **Cleanup:** When the initial scan for a '1' in the multiplier finds the $*$ separator instead, it means all digits of the multiplier have been processed. The machine then enters a final cleanup phase, moving back and forth across the tape to erase all X s, the $*$, and the original multiplicand's '1's, leaving only the final product.
7. **Halt:** The machine halts with $m * n$ ones on the tape.

The stark difference in the complexity of these two algorithms is revealing. Addition is a linear-time, single-pass operation requiring only a few states. Multiplication, with its nested-loop structure, requires a significantly larger state machine and runs in polynomial time. This demonstrates how a seemingly modest increase in the abstract problem's difficulty translates into a substantial increase in the computational resources (states, transitions, and time) required

by the Turing machine. The visualizer will make this difference palpable, contrasting the simple sweep of addition with the methodical, repetitive scanning of multiplication.

3.2.2 Formal Transition Table

The intricate logic of the multiplication algorithm is encoded in the transition table below (Table 3.2). This table is the definitive blueprint for the multiplication TM, translating the high-level strategy into the machine's primitive instruction set.⁹

Table 3.2: State Transition Table for Unary Multiplication

State	Read	Write	Move	Next State	Description
q_0	I	X	R	q_1	Mark a '1' in multiplier m; start outer loop.
q_0	$*$	$*$	R	q_5	No more '1's in m; proceed to cleanup.
q_1	I	I	R	q_5	Move right to find the separator.
q_1	$*$	$*$	R	q_2	Found separator; start inner loop.
q_2	I	Y	R	q_3	Mark a '1' in multiplicand n; prepare to copy.
q_2	B	B	L	q_4	End of n reached; reset multiplicand.
q_2	Y	Y	R	q_2	Skip already-copied '1's.
q_3	I	I	R	q_3	Move right to find the end of the tape.
q_3	B	I	L	q_2	Write a '1' in result area; return to n.
q_4	Y	I	L	q_4	Reset a 'Y' back to '1'.
q_4	$*$	$*$	L	q_4	Move left past separator.
q_4	I	I	L	q_4	Move left past '1's in m.
q_4	X	X	R	q_0	Find start of m; begin next outer loop iteration.
q_5	I	B	R	q_5	Cleanup: Erase '1's of n.
q_5	Y	B	R	q_5	Cleanup: Erase any leftover 'Y's.

q_5	B	B	L	q_6	Finished erasing right side; move left.
q_6	l	B	L	q_6	Move left, erasing result as we go.
q_6	$*$	B	L	q_7	Erase separator.
q_7	X	B	L	q_7	Erase 'X's of m.
q_7	B	B	R	q_f	Cleanup complete; halt.

Section 4: System Architecture and Implementation

4.1 Core Simulator Engine (C++)

The backend engine is an object-oriented C++ implementation of the formal Turing machine model.

4.1.1 Data Structures

The choice of data structures is critical for an efficient and conceptually clean implementation.

- **Tape:** The tape is represented by a **deque<char>**. This container is chosen over **vector** or **list** for its specific performance characteristics. A deque (double-ended queue) provides amortized constant time, $O(1)$, for insertions and deletions at both its beginning and end.¹⁸ This is essential for simulating an infinitely extendable tape; when the head moves off one end, a new blank cell can be added efficiently without requiring a costly reallocation and copy of the entire tape's contents, as would be the case with **vector**.
- **Head:** The head's position is managed by an integer index, **int head_pos**, which points to the current cell within the **deque**.
- **Transition Function:** The transition function is implemented using a **map**. The key for the map is a **pair<int, char>**, representing the current state and the symbol read from the tape. The value is a **tuple<int, char, int>**, representing the new state, the symbol to write, and the direction to move (e.g., -1 for Left, 1 for Right). The **map** provides a clean and readable way to store the transition rules and offers efficient logarithmic time, $O(\log N)$, lookup for the next action, where N is the number of rules.¹⁹

4.1.2 The TuringMachine Class

The entire state and logic of the simulation are encapsulated within a TuringMachine class. Its primary members include:

- deque<char> tape;
- int head_pos;
- int current_state;
- map<pair<int, char>, tuple<int, char, int>> transitions;
- bool halted;

The class provides a public interface with methods to control the simulation:

- void loadMachine(const auto& rules, const string& input, int start_state);: Initializes the machine with a set of transition rules, an initial tape string, and a starting state.
- void step();: Executes a single computational step.
- void run();: Executes steps repeatedly until the machine halts.
- void reset();: Resets the machine to its initial configuration.

4.1.3 The Simulation Loop

The core logic resides within the `step()` method. In each invocation, it performs the following sequence:

1. Check if the machine is already in a halted state. If so, do nothing.
2. Read the symbol at the current `head_pos` on the tape. If the `head_pos` is out of bounds (which can happen on the first move to a new blank cell), expand the deque by adding a blank symbol.
3. Construct a key (`current_state`, `symbol_read`).
4. Search for this key in the transitions map.
5. If a transition is found:
 - a. Retrieve the (`new_state`, `write_symbol`, `direction`) tuple.
 - b. Write `write_symbol` to the tape at `head_pos`.
 - c. Update `current_state` to `new_state`.
 - d. Update `head_pos` by adding `direction` (-1 or +1).
6. If no transition is found: Set the halted flag to true.

4.2 The Graphical User Interface (GUI)

The GUI provides a visual window into the machine's execution, allowing for interactive control and observation.

4.2.1 Choice of Framework: Dear ImGui

The project utilizes **Dear ImGui** as its GUI framework. This choice is deliberate and philosophically aligned with the nature of the simulation. Unlike traditional "retained mode" GUI toolkits like Qt or wxWidgets, which build and maintain a persistent hierarchy of UI objects, Dear ImGui is an "immediate mode" library.²⁰ In an immediate mode paradigm, the entire UI is conceptually redrawn from scratch every frame based on the application's current data state.²⁰

This approach is exceptionally well-suited for a visualizer. The `TuringMachine` object serves as the single source of truth. The GUI does not need to maintain its own complex state (e.g., the text in each tape cell widget). Instead, each frame, the application simply iterates through the `TuringMachine`'s tape deque and issues draw commands to display its contents. This eliminates a vast category of potential bugs related to keeping the UI and the backend simulation synchronized.²⁰ This architectural synergy, where the "stateless" UI paradigm mirrors the discrete, state-based operation of the Turing machine itself, results in a simpler, more robust, and more elegant implementation.

4.2.2 UI Components Design

The UI layout is designed for clarity and ease of use, drawing inspiration from established online simulators.⁸

- **Tape Display:** A central component is a horizontally scrollable region displaying the tape. Each cell is rendered as a bordered box containing its character. The view is

programmatically centered on the head, ensuring the active part of the tape is always visible.

- **Head Indicator:** The tape cell currently under the head is highlighted with a distinct background color and a pointer arrow above it to make its position unambiguous.
- **Status Panel:** A simple panel displays key information in real-time: Current State, Steps Executed, and a Status indicator (e.g., "Running," "Halted").
- **Control Panel:** An interactive panel contains:
 - An InputText field for the user to enter the initial tape string.
 - Buttons to Load Addition TM and Load Multiplication TM, which populate the transitions map in the backend.
 - Execution controls: Run, Pause, Step, and Reset.
 - A slider to control the execution speed in Run mode.

4.2.3 Backend and Frontend Integration

The main application loop orchestrates the interaction between the simulation engine and the GUI, following standard Dear ImGui practice.²⁷

```
// Simplified main loop
while (!window_should_close) {
    // 1. Start a new ImGui frame
    ImGui_ImplOpenGL3_NewFrame();
    ImGui_ImplGlfw_NewFrame();
    ImGui::NewFrame();

    // 2. Handle GUI interaction and update backend state
    // (e.g., if (ImGui::Button("Step")) { tm.step(); })
    // (e.g., if (running && !paused) { tm.step(); })

    // 3. Draw the UI by reading from the backend state
    // (e.g., draw_tape_widget(tm.getTape(), tm.getHeadPos());)
    // (e.g., ImGui::Text("Current State: %d", tm.getCurrentState());)

    // 4. Render the ImGui frame
    ImGui::Render();
    ImGui_ImplOpenGL3_RenderDrawData(ImGui::GetDrawData());

    // 5. Swap graphics buffers
    glfwSwapBuffers(window);
}
```

This structure ensures a clean separation of concerns. The TuringMachine class knows nothing about the GUI, and the GUI code simply reads data from the TuringMachine object to render the view for that frame.

Section 5: Verification and Testing

A systematic testing plan is crucial to verify the correctness of both the Turing machine algorithms and their C++ implementation. The methodology combines backend unit tests with frontend, user-driven validation, leveraging the visualizer as a primary debugging tool.

5.1 Test Plan and Methodology

The testing process is twofold:

1. **Backend Verification:** The core TuringMachine class is tested in isolation. Test cases involve loading the transition tables from Section 3, initializing the tape with specific inputs, running the simulation, and asserting that the final tape state matches the expected output.
2. **Frontend Validation:** The full application is tested interactively. This involves using the GUI to load machines, input test strings, and visually inspect the step-by-step execution and the final result.

The primary success criterion for any test case is that the machine must halt on the correct final tape configuration, which should contain only the arithmetic result, with all original operands and auxiliary markers erased and replaced by blank symbols.

5.2 Test Cases for Unary Addition

The addition machine is tested against a set of base and edge cases to ensure its robustness. The input format is $1^m + 1^n$.

- **Base Cases:**
 - $2 + 3 \rightarrow$ Input: 11+111, Expected Output: 11111
 - $4 + 3 \rightarrow$ Input: 1111+111, Expected Output: 1111111
 - $1 + 1 \rightarrow$ Input: 1+1, Expected Output: 11
- **Edge Cases:**
 - **Addition with Zero (First Operand):** $0 + 3 \rightarrow$ Input: +111, Expected Output: 111
 - **Addition with Zero (Second Operand):** $3 + 0 \rightarrow$ Input: 111+, Expected Output: 111
 - **Symmetry Check:** Verify that the result of $2 + 3$ is identical to $3 + 2$.

5.3 Test Cases for Unary Multiplication

The more complex multiplication machine requires more rigorous testing, particularly around edge cases involving zero and one. The input format is $1^m * 1^n$.

- **Base Cases:**
 - $2 * 3 \rightarrow$ Input: 11*111, Expected Output: 111111⁹
 - $3 * 2 \rightarrow$ Input: 111*11, Expected Output: 111111
- **Edge Cases:**
 - **Multiplication by One (First Operand):** $1 * 4 \rightarrow$ Input: 1*1111, Expected Output: 1111

- **Multiplication by One (Second Operand):** $4 * 1 \rightarrow$ Input: 1111*1, Expected Output: 1111
- **Multiplication by Zero (First Operand):** $0 * 3 \rightarrow$ Input: *111, Expected Output: (empty tape)
- **Multiplication by Zero (Second Operand):** $3 * 0 \rightarrow$ Input: 111*, Expected Output: (empty tape)
- **Zero by Zero:** $0 * 0 \rightarrow$ Input: *, Expected Output: (empty tape)
- **One by One:** $1 * 1 \rightarrow$ Input: 1*1, Expected Output: 1

These zero-related cases are critical as they test the machine's ability to correctly handle empty loops and proceed directly to the cleanup phase.

5.4 Analysis of Results and Debugging

During development, the visualizer proved to be an indispensable debugging tool. For the multiplication machine, initial implementations produced incorrect output for certain edge cases. For example, an early version failed on $3*0$ because the transition from state q_0 did not properly account for immediately reading the $*$ symbol.

Instead of relying on print statements or a traditional debugger, the error was diagnosed by loading the faulty machine, inputting 111*, and using the Step button. It became immediately apparent that the machine entered an undefined state after the initial scan. By observing the Current State display and the head's position frame-by-frame, the exact point of failure was identified. This allowed for a targeted correction to the transition table (adding rule for $(q_0, *)$ to $(q_5, *, R)$), demonstrating the power of visualization in making the abstract logic of the machine transparent and debuggable.

Section 6: Evaluation and Future Work

6.1 Project Evaluation

The project successfully met the objectives outlined in Section 1.3.

1. **Algorithmic Design:** Formal algorithms and transition tables for unary addition and multiplication were successfully designed and documented. The implementation of these tables demonstrated the Turing machine's capacity to perform non-trivial computations using only primitive operations.
2. **Visual Simulator Implementation:** A functional and interactive visualizer was developed in C++ using Dear ImGui. The GUI provides a clear, real-time representation of the machine's configuration (tape, head, state) and allows for fine-grained control over the simulation (run, step, pause, reset). It serves effectively as a pedagogical tool, transforming the abstract theory of TM execution into an observable process.

The project highlights a crucial aspect of theoretical computer science: the limitations of a computational model are not failures, but rather its most defining and insightful features. The simulator's "flaws"—its practical inefficiency and its inability to solve the Halting Problem—are direct, tangible manifestations of deep theoretical truths. The slowness on large inputs is not a sign of poor C++ code but a visible demonstration of algorithmic complexity. The potential for an infinite loop is not a bug but a live encounter with the concept of undecidability. In this way, the simulator becomes an interactive laboratory for exploring the very boundaries of computation.

6.2 Limitations of the Simulator

The simulator has both practical limitations stemming from its implementation on physical hardware and theoretical limitations inherited from the Turing machine model itself.

- **Practical Limitations:**
 - **Finite Memory:** The foundational concept of an infinite tape is approximated by a dynamically sized deque. This data structure is ultimately bounded by the available system RAM, a crucial distinction between the abstract model and any physical realization.²
 - **Performance:** The Turing machine is a model of computability, not efficiency.² The step-by-step simulation, especially for the polynomial-time multiplication algorithm, becomes noticeably slow for large inputs (e.g., 50×50). This slowness, coupled with the overhead of GUI rendering, demonstrates the model's impracticality for real-world computation, which relies on more efficient architectures like the random-access machine.¹
- **Theoretical Limitations:**
 - **The Halting Problem:** The simulator makes no attempt to solve the Halting Problem. It is fundamentally impossible to create a universal algorithm that can determine, for an arbitrary Turing machine and its input, whether that machine will ever halt.³⁰ If the simulator is loaded with a transition table that contains an infinite loop (e.g., a rule like

$(q_0, 1)$ to $(q_0, 1, R)$ on a tape of all '1's), it will run forever. This is not a flaw in the simulator but a correct simulation of a non-halting machine.

6.3 Potential Enhancements and Future Work

The current simulator provides a solid foundation that could be extended in several compelling directions to further explore the landscape of theoretical computer science.

- **Expanded Algorithm Library:** The simulator could be enhanced by implementing a wider range of classical TM algorithms, such as unary subtraction and division, palindrome recognition, string sorting, or language recognizers (e.g., for $L = 0^n 1^n \mid n \geq 1$).
- **Multi-Tape Turing Machines:** An extension to simulate multi-tape Turing machines would be a significant enhancement. While any multi-tape TM can be simulated by a single-tape TM, the former is often far more efficient and algorithmically intuitive for certain problems, such as string copying or reversal.¹ This would allow for a visual comparison of single-tape vs. multi-tape efficiency.
- **Non-Determinism:** A major theoretical extension would be to add support for non-deterministic Turing machines (NTMs). This would require a more complex simulation engine capable of exploring multiple computation branches and a GUI that can visualize this branching computation tree.
- **Subroutine and Composition Visualization:** A powerful feature would be the ability to define TMs as subroutines that can be "called" by other TMs.³⁵ Recent research has focused on tools to visualize such compositions, helping students understand how complex machines can be built modularly, much like functions in a programming language.³⁶
- **Dynamic State Diagram Generation:** The GUI could be improved to parse the loaded transition table and dynamically generate a visual state transition diagram. During simulation, the currently active state and transition arc could be highlighted, providing a second layer of visualization that connects the tape's behavior directly to the machine's underlying program logic.⁸

Section 7: Conclusion

7.1 Summary of Findings

This project successfully navigated the path from the abstract, formal definition of a Turing machine to a concrete, interactive C++ simulator. By focusing on the fundamental arithmetic operations of unary addition and multiplication, it was demonstrated how this minimalist computational model can execute complex, stateful algorithms. The design process translated high-level procedural descriptions into rigorous, unambiguous state transition tables, which were then implemented in a robust C++ engine. The selection of an immediate-mode GUI framework, Dear ImGui, proved to be an architecturally sound choice, creating a clean and responsive interface that directly reflects the state of the backend simulation. The final application serves as an effective pedagogical tool, making the often-opaque mechanics of Turing machine computation visible and intuitive.

7.2 Final Reflections

The Turing machine remains, nearly a century after its conception, the bedrock of theoretical computer science. While its minimalist design makes it inefficient for practical, real-world computing, its simplicity is its greatest theoretical strength. It provides a universal standard against which the very limits of algorithmic computation can be measured. This project, in creating a visualizer for this foundational model, serves as a testament to this legacy. By observing the simple, mechanical dance of the read/write head as it transforms a string of symbols, we gain a more profound appreciation for the nature of computation itself. Understanding the operations of the simplest possible computer provides invaluable insight into the capabilities, complexities, and fundamental limitations of all computers, past, present, and future.

References

1. J. Erickson, “*Turing Machines*,” University of Illinois, 2019. [Online]. Available: <https://jeffe.cs.illinois.edu>
2. “*Turing machine*,” Wikipedia, 2025. [Online]. Available: https://en.wikipedia.org/wiki/Turing_machine
3. P. E. Griffiths, “*Turing Machines*,” *The Stanford Encyclopedia of Philosophy*, Stanford University, 2023. [Online]. Available: <https://plato.stanford.edu/entries/turing-machine/>
4. “*Definition of a Turing Machine*,” Cornell University, Department of Computer Science. [Online]. Available: <https://www.cs.cornell.edu/courses/cs4810/2014sp/TuringMachines.html>
5. “*Turing Machine Introduction*,” TutorialsPoint, 2024. [Online]. Available: https://www.tutorialspoint.com/automata_theory/turing_machine_introduction.htm
6. “*Turing Machines*,” Stanford University, Department of Computer Science. [Online]. Available: <https://web.stanford.edu/class/cs103/lectures/08/>
7. “*Turing Machine Visualization*,” TuringMachine.io, 2024. [Online]. Available: <https://turingmachine.io>
8. “*Turing Machine for Addition in Automata Theory*,” TutorialsPoint, 2024. [Online]. Available: https://www.tutorialspoint.com/automata_theory/turing_machine_for_addition.htm
9. “*Turing Machine for Addition*,” GeeksforGeeks, 2024. [Online]. Available: <https://www.geeksforgeeks.org/turing-machine-for-addition/>
10. “*Unary Representation of Numbers*,” Humanities LibreTexts, 2023. [Online]. Available: <https://human.libretexts.org>
11. J. McCarthy, “*Addition on Turing Machines*,” 2013. [Online]. Available: <https://jeapostrophe.github.io/2013-10-29-turing-machine-addition.html>
12. “*Turing Machine Simulator*,” morphett.info, 2024. [Online]. Available: https://morphett.info/turing_machine
13. “*Turing Machines - Computer Science: An Interdisciplinary Approach*,” Princeton University, 2024. [Online]. Available: <https://introcs.cs.princeton.edu/java/74turing/>
14. “*Turing-Machine-Simulator*,” GitHub Repository, projectavi, 2024. [Online]. Available: <https://github.com/projectavi/Turing-Machine-Simulator>
15. “*Turing Machines and Subroutine Simulation*,” Theoretical Computer Science Stack Exchange, 2024. [Online]. Available: <https://cstheory.stackexchange.com/questions/>